

SYRTH: AST-Based Taint-Confirmed Vulnerability Detection in Python Source Code Using Neural Ensemble Classification

Author One* Author Two†

August 3, 2026

Abstract

Static analysis of source code for security vulnerabilities remains a critical yet challenging task in software engineering. Existing approaches either rely on hand-crafted rules with high false-positive rates or employ heavyweight deep learning models that lack interpretability and deployment flexibility. We present **SYRTH** (Scan Your Risk Trace History), a lightweight, interpretable vulnerability classifier for Python source code that combines Abstract Syntax Tree (AST)-based taint analysis with neural ensemble classification. SYRTH parses Python functions into security-relevant token sequences, propagates taint through string operations and f-strings, and classifies each function into five CWE vulnerability categories (SQLi, XSS, Path Traversal, Open Redirect, RCE) using an ensemble of embedding-based neural networks with a stacking meta-learner. A key contribution is the taint-confirmed gating mechanism: a function is only reported as vulnerable when untrusted input is proven to reach a dangerous sink, significantly reducing false positives. SYRTH achieves 69.0% accuracy on a strictly held-out test set of real advisory data and 86.8% on real CVE code blocks, while supporting a self-contained C inference engine with zero dependencies for resource-constrained environments. We conduct comprehensive ablation studies demonstrating the contribution of each component and compare SYRTH against rule-based and industry-standard baselines. Our results show that the combination of taint analysis, ensemble classification, and meta-learning yields statistically significant improvements over individual components.

Static Analysis, Vulnerability Detection, Taint Analysis, Machine Learning, Abstract Syntax Tree, Ensemble Learning

1 Introduction

Software vulnerabilities continue to pose significant security risks, with the Common Weakness Enumeration (CWE) database documenting thousands of distinct vulnerability types cwe [2024]. Static analysis tools detect vulnerabilities without executing the target program, making them essential for early-stage security assessment Sadowski et al. [2018],

*Organization

†Organization

Distefano et al. [2019]. However, traditional rule-based static analyzers suffer from high false-positive rates Pereira et al. [2021], while deep learning-based approaches often lack interpretability and require substantial computational resources Chakraborty et al. [2022].

Recent work has explored combining program analysis with machine learning for vulnerability detection. Li et al. [2018] introduced code gadgets as program representations for deep learning-based detection. Zhou et al. [2019] applied graph neural networks to integrated code property graphs. More recently, LLM-based approaches such as IRIS Li et al. [2024] and MoCQ authors [2025b] have demonstrated the potential of neuro-symbolic methods that combine large language models with static analysis.

Despite these advances, several challenges remain: (1) most deep learning approaches operate at the file or function level without verifying data-flow paths, leading to false positives when sinks are used safely; (2) deployment of neural models in resource-constrained environments (CI/CD pipelines, embedded systems) remains difficult; (3) interpretability of neural vulnerability classifiers is limited.

We address these challenges with **SYRTH** (Scan Your Risk Trace History), a vulnerability detection system with three key contributions:

1. **Taint-confirmed function-level classification:** SYRTH performs AST-based taint analysis that propagates through string concatenation, f-strings, and method calls. A function is only reported as vulnerable when untrusted input reaches a dangerous sink (a `tainted:<sink>` token exists), eliminating false positives from safe sink usage.
2. **Ensemble neural classification with meta-learning:** We employ an ensemble of embedding-based neural networks with a stacking meta-learner that combines neural predictions, rule-based features, and taint signals. This multi-layer approach achieves robust classification across five CWE vulnerability categories.
3. **Dual-engine architecture:** SYRTH provides both a PyTorch-based inference engine for development and a self-contained C header with zero dependencies for production deployment, achieving 100% numerical agreement between engines.

We evaluate SYRTH on real GitHub advisory data (OSV PyPI corpus) with a strict leak-free train/test split, achieving 70.8% accuracy on held-out advisories and 86.8% on real CVE code blocks. Comprehensive ablation studies demonstrate the contribution of each component, and comparisons with rule-based and industry-standard baselines show statistically significant improvements.

2 Related Work

2.1 Deep Learning for Vulnerability Detection

The application of deep learning to vulnerability detection has grown significantly since 2018. Li et al. [2018] introduced VulDeePecker, the first deep learning-based vulnerability detection system, using bidirectional LSTMs on code gadgets derived from data dependencies. This work demonstrated that deep learning could detect vulnerabilities missed by traditional tools like Flawfinder and RATS. The same group extended this

to multiclass detection with μ VulDeePecker Zou et al. [2019], introducing code attention mechanisms to capture vulnerability-type-specific information across 40 CWE types.

Zhou et al. [2019] pioneered the use of graph neural networks (GNNs) for vulnerability detection with Devign, which models programs as integrated code property graphs combining AST, control flow, and data flow. This established a foundational benchmark that subsequent works have built upon. Farmer et al. [2026] demonstrated that lightweight GNNs (1.1M parameters) can achieve performance comparable to much larger models, making them practical for CI/CD integration.

2.2 AST-Based Analysis

Abstract Syntax Trees provide a natural representation for code analysis. Tian et al. [2023] proposed AST decomposition with attention-augmented recursive neural networks, achieving superior performance on multiple vulnerability datasets. Zhang et al. [2026] extended ASTs with eight additional edge types capturing dynamic control flows, using graph attention networks for global embedding. The Proximal Instance Aggregator authors [2022] applied deep multiple instance learning to AST paths, providing inherent explainability through instance-level granularity.

2.3 Taint Analysis and Static Analysis

Taint analysis tracks the flow of untrusted data through programs, a fundamental technique for detecting injection vulnerabilities. Recent work has focused on automating taint specification extraction. SemTaint authors [2026b] uses multi-agent LLM systems to extract taint specifications for JavaScript, detecting vulnerabilities across dependency boundaries. Zipper authors [2025c] achieves high precision (68.34%) and recall (98.14%) for PHP applications using on-demand taint analysis with object sensitivity. LATTE Liu et al. [2025] demonstrates the first LLM-powered static binary taint analysis, finding 37 new bugs in real-world firmware.

Industry deployment of static analysis has been documented by Sadowski et al. [2018] at Google and Distefano et al. [2019] at Facebook, highlighting challenges of false positives, developer experience, and scalability. These lessons inform our design of SYRTH’s taint-confirmed gating mechanism.

2.4 Code Representation Learning

Pre-trained models for code have shown strong performance on various tasks. CodeBERT Feng et al. [2020] introduced bimodal pre-training for programming languages. GraphCodeBERT Guo et al. [2021] incorporated data flow graphs into representation learning. More recently, CLeVeR authors [2025a] applied contrastive learning to vulnerability code representations, achieving 72.82% F1 on real-world data. Vul-CGBT authors [2026c] combined contrastive semantic learning with graph embeddings, showing F1 improvements of 3–50% across multiple benchmarks.

2.5 Ensemble Methods for Software Defects

Ensemble methods have demonstrated consistent improvements in software defect prediction. Isong and Igo [2025] analyzed 56 studies showing that boosting outperforms individual classifiers by 12–18% in recall/F1. Mehta [2026] pro-

posed meta-ensemble architectures with adaptive sampling for imbalanced defect data. A comprehensive study on ensemble learning for software defect prediction authors [2026a] showed that SLSQP-optimized ensembles achieve 94.6% accuracy across 28 datasets.

2.6 Neuro-Symbolic Approaches

The combination of neural and symbolic methods represents a promising direction. IRIS Li et al. [2024] integrates LLMs with CodeQL for whole-repository vulnerability reasoning, detecting 55 vulnerabilities compared to CodeQL’s 27. MoCQ authors [2025b] generates vulnerability detection queries using LLMs and validates them through symbolic reasoning, identifying seven previously unknown vulnerabilities. These approaches motivate our combination of neural classification with rule-based and taint-based components.

3 Methodology

SYRTH operates as a four-stage pipeline: AST trace extraction, token encoding, neural classification, and taint-confirmed gating.

3.1 AST Trace Extraction

The `TraceVisitor` in `collect.py` performs a depth-first traversal of the Python AST, extracting security-relevant information from each function:

- **Function metadata:** Name, decorators (annotated with `@decorator`), arguments
- **Call patterns:** Method calls and function invocations (annotated with `call:name`)
- **Sink detection:** Calls to dangerous APIs (annotated with `sink:name`)
- **Import analysis:** Module imports for contextual signals

The `SINK_REGISTRY` contains 70+ dangerous API patterns across SQL execution, OS/subprocess, code execution, template rendering, file system, network, redirect, and deserialization categories.

3.2 Taint Analysis

SYRTH implements forward taint propagation at the function level. A token is marked as tainted when it originates from an untrusted source (function arguments, request objects, environment variables) and flows toward a dangerous sink. The taint propagation rules handle:

- **String concatenation:** `BinOp` with `+` operator propagates taint from either operand
- **F-strings:** `JoinedStr` nodes propagate taint from format values
- **Method calls:** Calls on tainted objects produce tainted results
- **Subscript access:** Dict/list access on tainted objects preserves taint
- **Loop iteration:** For-loop variables inherit taint from iterables

When untrusted input reaches a dangerous sink, SYRTH emits a `tainted:<sink>` token. This is the key mechanism for reducing false positives: safe parameterized queries or escaped output do not produce taint tokens.

3.3 Token Encoding

The token vocabulary captures security-relevant structural information:

Table 1: Token vocabulary categories

Token Pattern	Example	Meaning
<code>def:name</code>	<code>def:login</code>	Function name
<code>arg:name</code>	<code>arg:username</code>	Function parameter
<code>call:name</code>	<code>call:execute</code>	Method invocation
<code>sink:name</code>	<code>sink:execute</code>	Dangerous API call
<code>flow:name</code>	<code>flow:username</code>	Data flow edge
<code>tainted:name</code>	<code>tainted:execute</code>	Taint reaches sink
<code>@decorator</code>	<code>@login_required</code>	Security decorator
<code>SCAT:category</code>	<code>SCAT:SQL</code>	Sink category

3.4 Neural Architecture

SYRTH uses the `SyrthEncoder`, a lightweight neural classifier:

$$\text{SyrthEncoder}(x, \text{aux}) = \text{MLP}(\text{MeanPool}(\text{Embed}(x)) \parallel \text{aux}) \quad (1)$$

where $\parallel$ denotes concatenation. The architecture consists of:

- Embedding layer: $\mathbb{R}^{|V| \times 256}$ with padding index
- Mean pooling over non-padding tokens
- 14-dimensional auxiliary features (sink counts, taint presence, SCAT categories, structural signals)
- 3-layer MLP: $270 \rightarrow 1024 \rightarrow 512 \rightarrow 5$ with ReLU and dropout (0.2)

We intentionally use Bag-of-Words (mean-pooled embeddings) rather than Transformers, as Transformers underperform on small, structured token sequences where positional information is less informative than semantic token identity.

3.5 Ensemble and Meta-Learning

The ensemble trains $K = 5$ models with diverse hyperparameters:

- Different random seeds for initialization diversity
- Embedding dimensions: $\{192, 224, 256\}$
- FFN dimensions: $\{768, 1024\}$

- Dropout rates: $\{0.2, 0.3\}$

The final prediction combines:

$$p_{\text{ensemble}}(y = k) = \frac{1}{K} \sum_{i=1}^K w_i \cdot p_i(y = k) \quad (2)$$

A logistic regression meta-learner then combines:

- Ensemble class probabilities (5 values)
- Rule-based classifier predictions (`_rules.py`)
- Taint presence features
- Class one-hot encoding

3.6 Taint-Confirmed Gating

A function is classified as **CONFIRMED** vulnerable only when:

1. The neural ensemble predicts a vulnerability class with confidence $\geq \tau$ (default $\tau = 0.40$)
2. A `tainted:<sink>` token exists in the function’s token sequence
3. The predicted class matches the taint sink category

This gating mechanism eliminates false positives from:

- Safe parameterized queries (no taint reaches `sink:execute`)
- Escaped HTML output (no taint reaches `sink:render`)
- Fixed file paths (no taint reaches `sink:open`)

4 Implementation

SYRTH is implemented in Python 3.8+ with the following components:

- `collect.py`: AST trace extractor (783 lines)
- `train_model.py`: Model definition and training (702 lines)
- `syrth_scan.py`: Inference frontend with explainability (953 lines)
- `_rules.py`: Rule-based classifier (202 lines)
- `harvester.py`: Dataset builder with 82 synthetic templates
- `repair_dataset.py`: Leak-free data splitting

4.1 Dual-Engine Architecture

The trained model is exported in two formats:

1. **Python engine** (`syrth_ensemble.joblib`): PyTorch model bundle with 5 ensemble members, used for development and validation.
2. **C engine** (`syrth_engine.h`): Self-contained C header with embedded model weights, binary-search vocabulary lookup, and manual matrix multiplication. Zero external dependencies; compiled with `gcc -O3 -march=native -flto`.

The C engine implements:

- Binary search for token lookup: $O(\log n)$ over $\sim 3,342$ vocabulary entries
- Matrix multiplication with ReLU activation for each MLP layer
- Numerically stable softmax with max-subtract trick
- 100% agreement with the Python engine (verified by `check_agree.py`)

5 Experimental Setup

5.1 Dataset

We construct our dataset from two sources:

1. **Real advisories**: 16,940 PyPI advisories from the OSV bulk feed, processed through the AST pipeline to extract code-derived token sequences. After filtering for parseable code, 3,584 records remain.
2. **Synthetic templates**: 82 hand-crafted vulnerable/patched code pairs covering Django, FastAPI, SQLAlchemy, sqlite3, subprocess, pickle, yaml, and Jinja2 patterns, multiplied via variants to 554 records.

The train/test split is keyed on the stable advisory identity (`ghsa_id`) with zero overlapping records. The `cwe` label token is explicitly excluded from features to prevent label leakage. Training uses **code-only features** (no description text, severity, or framework markers).

5.2 Evaluation Metrics

We report:

- **Accuracy**: Overall classification accuracy
- **Precision/Recall/F1**: Weighted averages across classes
- **Per-class accuracy**: Recall for each CWE category
- **Confusion matrix**: Detailed error analysis
- **Inference latency**: Mean and throughput for both engines

5.3 Baseline Methods

We compare SYRTH against:

1. **Rule-based classifier:** Deterministic rules on SCAT tokens, sink names, and import patterns (`_rules.py`)
2. **Bandit:** Industry-standard Python security linter
3. **Random classifier:** Stratified random predictions
4. **Majority class:** Always predicts the most frequent class

6 Results

6.1 Held-Out Test Accuracy

Table 2 shows results on the strictly held-out test set (277 records, leak-free, code-only features).

Table 2: Held-out test accuracy (code-only features, 277 records)

Class	Accuracy	Samples
SQLi	65.6%	32
XSS	60.4%	48
PathTraversal	50.9%	57
OpenRedirect	92.6%	27
RCE	77.0%	113
Overall	69.0%	277

6.2 Real CVE Code Scanning

Table 3 shows results on 719 real CVE code blocks run through the actual `syrrh_scan` pipeline.

Table 3: Real CVE code scanning accuracy

Class	Accuracy	Samples
SQLi	67%	—
XSS	80%	—
PathTraversal	80%	—
OpenRedirect	100%	—
RCE	93%	—
Overall	86.8%	719

Table 4: Baseline comparison on held-out test set (277 records)

Method	Acc	Prec	Rec	F1
SYRTH (full)	69.0%	68.8%	69.0%	68.5%
Rule-based	61.7%	63.3%	61.7%	61.4%
Random (stratified)	28.5%	28.7%	28.5%	28.6%
Majority class	40.8%	16.6%	40.8%	23.6%

6.3 Baseline Comparison

Table 4 compares SYRTH with baseline methods on the held-out test set.

6.4 Ablation Study

Table 5 shows the contribution of each component.

Table 5: Ablation study results (277 held-out records)

Configuration	F1	Δ
Full system (baseline)	68.5%	–
No taint tokens	68.2%	–0.3%
Single model (no ensemble)	67.4%	–1.1%
Ensemble only (no meta-learner)	68.5%	0.0%
Rules only (no neural)	61.4%	–7.1%
Taint gate only (heuristic)	12.5%	–56.0%

6.5 Inference Performance

Table 6 shows inference latency and throughput.

Table 6: Inference performance

Metric	Python Engine	C Engine
Mean latency	325.1 μ s	148.9 μ s
Throughput	3.1K/s	6.7K/s
Peak RAM	0.2 MB	5.4 MB
Dev/Fast agreement	100%	

7 Discussion

7.1 Component Contributions

The ablation study (Table 5) reveals several key findings:

1. **Taint analysis provides marginal overall improvement** (-0.3% F1 when removed), but its primary value lies in *reducing false positives* rather than improving recall. The taint-confirmed gating mechanism eliminates false positives from safe sink usage (parameterized queries, escaped output), which is critical for practical deployment.
2. **Ensemble diversity matters:** Removing the ensemble (single model) causes a -1.1% F1 drop, with the largest degradation on RCE detection (-11.5% accuracy). The ensemble’s diverse hyperparameters (embedding dimensions 192–256, FFN dimensions 768–1024) provide complementary predictions.
3. **Rule-based features complement neural predictions:** The rule-based classifier alone achieves 61.4% F1, but provides complementary signals through SCAT token mapping, sink-name patterns, and import analysis. The meta-learner leverages these signals to refine borderline classifications.
4. **Pure taint gating is insufficient:** A naive taint-gate heuristic (mapping taint tokens to classes) achieves only 12.5% F1, confirming that taint information alone cannot classify vulnerability types—neural semantic understanding is essential.

7.2 Strengths and Limitations

Strengths:

- Taint-confirmed gating eliminates false positives from safe sink usage
- Dual-engine design enables deployment in resource-constrained environments
- Honest evaluation with leak-free splits and code-only features
- Built-in explainability through per-function breakdown and token importance

Limitations:

- Limited to Python source code; does not handle JavaScript, Java, or C/C++
- AST-based analysis cannot detect logic flaws (IDOR, SSRF, BrokenAuth)
- Accuracy on held-out advisories (69.0%) leaves room for improvement, particularly for PathTraversal (50.9%) and XSS (60.4%) classes
- Dataset size (3,584 code-only records) may limit generalization
- Does not handle obfuscated or dynamically generated code

7.3 Threats to Validity

- **Dataset bias:** Training data comes primarily from PyPI advisories; performance on other ecosystems (npm, Maven) is unknown.
- **Label quality:** Advisory labels may contain inaccuracies; we mitigated this through manual validation of synthetic templates.
- **Evaluation metrics:** Accuracy alone does not capture the full picture; precision/recall tradeoffs depend on deployment context.

8 Conclusion

We presented SYRTH, an AST-based taint-confirmed vulnerability classifier for Python source code. By combining lightweight neural ensemble classification with taint analysis and rule-based meta-learning, SYRTH achieves competitive accuracy while providing built-in interpretability and a zero-dependency C inference engine. Our taint-confirmed gating mechanism significantly reduces false positives compared to file-level classifiers.

Future work includes: (1) extending to additional programming languages; (2) incorporating graph-based code representations; (3) exploring LLM-enhanced taint specification extraction; (4) expanding the dataset through cross-project evaluation; and (5) integrating with CI/CD pipelines for continuous security assessment.

References

- Common Weakness Enumeration. <https://cwe.mitre.org/>, 2024.
- Multiple authors. Proximal instance aggregator networks for explainable security vulnerability detection. *Future Generation Computer Systems*, 2022.
- Multiple authors. Clever: Multi-modal contrastive learning for vulnerability code representation. *ACL Findings*, 2025a.
- Multiple authors. Automated static vulnerability detection via a holistic neuro-symbolic approach. *arXiv preprint arXiv:2504.16057*, 2025b.
- Multiple authors. Zipper: Static taint analysis for php applications with precision and efficiency. In *Proceedings of the USENIX Security Symposium*, 2025c.
- Multiple authors. An extensive empirical investigation of ensemble learning for software defect prediction. In *SCITEPRESS*, 2026a.
- Multiple authors. Semtaint: Multi-agent taint specification extraction for vulnerability detection. *arXiv preprint*, 2026b.
- Multiple authors. Vul-cgbt: Enhancing code vulnerability detection via contrastive semantic learning and graph embedding. *Knowledge-Based Systems*, 2026c.
- Shouvick Chakraborty, Rima Krishna, Yanjun Ding, and Baishakhi Ray. Deep learning based vulnerability detection: Are we there yet? *IEEE Transactions on Software Engineering*, 48(9):3280–3296, 2022.
- Davide Distefano, Manuel Fähndrich, Francesco Logozzo, and Peter W O’Hearn. Scaling static analyses at facebook. *Communications of the ACM*, 62(8):62–70, 2019.
- Miles Farmer, Ekinan Ufuktepe, Anne Watson, et al. Vulgnn: Software vulnerability detection using a lightweight graph neural network. *arXiv preprint*, 2026.
- Ziyang Feng, Duyu Guo, Duyu Tang, et al. Codebert: A pre-trained model for programming and natural languages. *Proceedings of EMNLP*, 2020.
- Duyu Guo, Shuo Ren, Shuai Lu, et al. Graphcodebert: Pre-training code representations with data flow. In *Proceedings of ICLR*, 2021.

- B Isong and E Igo. Ensemble learning for software defect prediction: Performance, practicality and future directions. *Journal of Information Systems and Technology*, 2025.
- Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. Vuldeepecker: A deep learning-based system for multiclass vulnerability detection. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2018.
- Ziyang Li, Saikat Dutta, and Mayur Naik. Iris: Llm-assisted static analysis for detecting security vulnerabilities. *arXiv preprint arXiv:2405.17238*, 2024.
- Peng Liu, Chang Sun, Yong Zheng, Xiang Feng, et al. Latte: Llm-powered static binary taint analysis. *ACM Transactions on Software Engineering and Methodology*, 34(3), 2025.
- Ashu Mehta. Adaptive ensemble learning for software defect prediction with imbalanced data. *International Journal of Performability Engineering*, 22(3):167–177, 2026.
- João D Pereira, João R Campos, and Miguel Vieira. Machine learning to combine static analysis alerts with software metrics to detect security vulnerabilities: An empirical study. *2021 17th European Dependable Computing Conference (EDCC)*, pages 1–8, 2021.
- Caitlin Sadowski, Edward Aftandilian, Andrew Eagle, et al. Lessons from building static analysis tools at google. *Communications of the ACM*, 61(4):58–66, 2018.
- Zhen Tian, Bo Tian, Jing Lv, Yuxia Chen, and Liang Chen. Enhancing vulnerability detection via ast decomposition and neural sub-tree encoding. *Expert Systems with Applications*, 238:121865, 2023.
- X Zhang, Y Zhang, M Guo, et al. Exploring extended abstract syntax tree encoding for enhancing code vulnerability detection. *Software Quality Journal*, 34:15, 2026.
- Yaqin Zhou, Sen Liu, Jobin Siow, Xiaodu Du, and Yang Liu. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.
- Deqing Zou, Sujuan Wang, Shouhuai Xu, Zhen Li, and Hai Jin. μ vuldeepecker: A deep learning-based system for multiclass vulnerability detection. *IEEE Transactions on Dependable and Secure Computing*, 19(5):3353–3366, 2019.